Kishita Ahuja

D15B 24

MLDL

Practical 3

Application of Decision Tree and Random Forest for Classification Tasks

---

1. Dataset Source

The dataset used for this experiment is the **Heart Disease Dataset**, which is a real-world medical dataset commonly used for classification problems.

Dataset Source (Kaggle Mirror):
https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

---

2. Dataset Description

The Heart Disease dataset contains medical information of patients used to predict whether a person Dataset Characteristics:

- Number of instances: 1025 records
- Number of attributes: 14
- Type: Structured dataset (Numerical + Categorical features)
- Problem Type: Binary Classification

Important Features

Feature	Description
age	Age of the patient
sex	Gender (1 = male, 0 = female)
cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar
thalach	Maximum heart rate achieved
exang	Exercise-induced angina

Target Variable

- **target**
    - 1 → Heart disease present
    - 0 → No heart disease

---

3. Mathematical Formulation of the Algorithms

Decision Tree Classifier

A Decision Tree classifier predicts the class by learning simple decision rules inferred from data features.

At each node, the best split is chosen using **Gini Index** or **Entropy**.

Gini Index:

$$\mathbf{Gini=1-\sum pi2}$$

Where:

- (p_i) is the probability of class (i)

The tree recursively splits the dataset until stopping conditions are met.

Random Forest Classifier

Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions.

Final Prediction=Majority Vote of all trees

Advantages:Advantages:

- Reduces overfitting
- Improves accuracy
- Handles high-dimensional data well

---

4. Algorithm Limitations

Decision Tree Limitations

- Prone to overfitting
- Sensitive to noise and small data changes
- Can become complex and hard to interpret

Random Forest Limitations

- Requires more computation and memory
- Less interpretable compared to a single decision tree
- Slower training for very large datasets

---

5. Methodology / Workflow

Steps Involved

1. Dataset collection from UCI/Kaggle
2. Data loading using Pandas
3. Feature selection and target identification
4. Train-test split (80% training, 20% testing)
5. Training Decision Tree classifier
6. Training Random Forest classifier
7. Model evaluation using accuracy and confusion matrix
8. Visualization of decision tree and feature importance

Workflow Diagram

Dataset → Preprocessing → Train-Test Split

→ Model Training → Prediction

→ Evaluation → Visualization

---

6. Performance Analysis

The performance of both models was evaluated using:

- **Accuracy Score**
- **Confusion Matrix**
- **Classification Report (Precision, Recall, F1-score)**

Observations

- Decision Tree provides good accuracy but may overfit the data.
- Random Forest achieves higher accuracy due to ensemble learning.
- Random Forest reduces variance and improves generalization.

Overall, Random Forest outperforms Decision Tree in terms of stability and prediction accuracy.

---

7. Hyperparameter Tuning

Decision Tree Parameters

- max_depth – controls tree depth
- min_samples_split – minimum samples required to split

Random Forest Parameters

- n_estimators – number of trees
- max_depth
- max_features

Tuning Impact

- Increasing tree depth improves accuracy but risks overfitting
- Increasing number of trees in Random Forest improves performance
- Proper tuning leads to better generalization on unseen data

---

Code:

```
# ========================================
# IMPORT LIBRARIES
# ========================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


# ========================================
# LOAD DATASET
# ========================================
# If using Google Drive (Colab)
# df = pd.read_csv('/content/drive/MyDrive/MLDL/heart.csv')
```

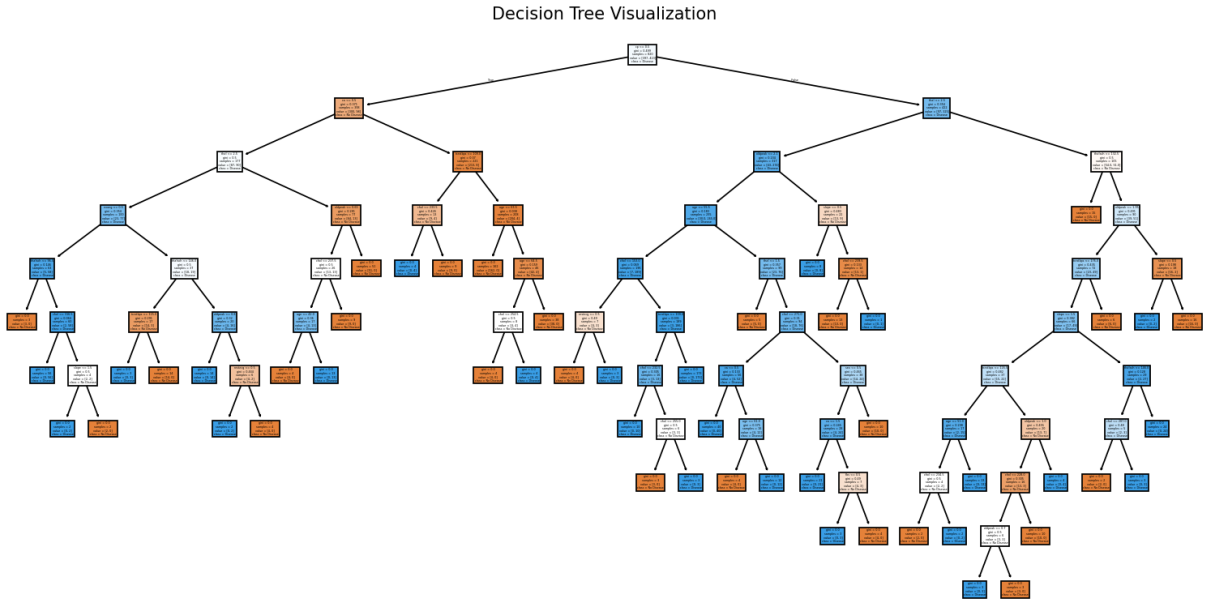
```
# If uploaded directly
df = pd.read_csv("heart.csv")

print(df.head())
print(df.shape)


# ========================================
# FEATURES & TARGET
# ========================================
X = df.drop('target', axis=1)
y = df['target']


# ========================================
# TRAIN-TEST SPLIT
# ========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# ========================================
# DECISION TREE CLASSIFIER
# ========================================
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

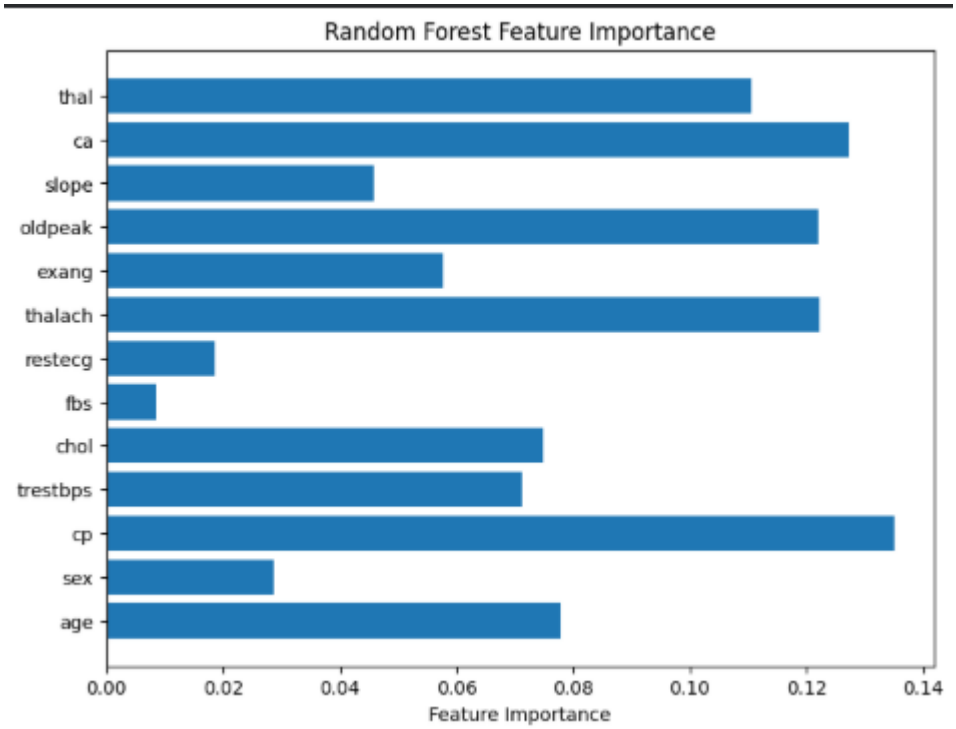
```
print("\n--- Decision Tree Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test, y_pred_dt))


# ========================================
# DECISION TREE VISUALIZATION
# ========================================
plt.figure(figsize=(20,10))
plot_tree(dt, feature_names=X.columns,
        class_names=["No Disease", "Disease"],
        filled=True)
plt.title("Decision Tree Visualization")
plt.show()


# ========================================
# RANDOM FOREST CLASSIFIER
# ========================================
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("\n--- Random Forest Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))

print("Classification Report:\n", classification_report(y_test, y_pred_rf))



# ========================================

# FEATURE IMPORTANCE (RANDOM FOREST)

# ========================================

importances = rf.feature_importances_


plt.figure(figsize=(8,6))

plt.barh(X.columns, importances)

plt.xlabel("Feature Importance")

plt.title("Random Forest Feature Importance")

plt.show()
```

OUTPUT:

```
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/vrkha/Desktop/MLDL/Prac3.py ================
   age  sex  cp  trestbps  chol  fbs  ...  exang  oldpeak  slope  ca  thal  target
0   52    1   0       125   212    0  ...      0      1.0      2   2     3       0
1   53    1   0       140   203    1  ...      1      3.1      0   0     3       0
2   70    1   0       145   174    0  ...      1      2.6      0   0     3       0
3   61    1   0       148   203    0  ...      0      0.0      2   1     3       0
4   62    0   0       138   294    1  ...      0      1.9      1   3     2       0

[5 rows x 14 columns]
(1025, 14)

--- Decision Tree Classifier ---
Accuracy: 0.9853658536585366
Confusion Matrix:
 [[102   0]
 [  3 100]]
Classification Report:
               precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103

    accuracy                           0.99       205
   macro avg       0.99      0.99      0.99       205
weighted avg       0.99      0.99      0.99       205


--- Random Forest Classifier ---
Accuracy: 0.9853658536585366
Confusion Matrix:
 [[102   0]
 [  3 100]]
Classification Report:
               precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103

    accuracy                           0.99       205
   macro avg       0.99      0.99      0.99       205
weighted avg       0.99      0.99      0.99       205

>>>
```





Conclusion

This experiment successfully demonstrates the application of Decision Tree and Random Forest algorithms for a real-world classification problem. The Random Forest classifier outperforms the Decision Tree due to its ensemble nature and robustness against overfitting, making it more suitable for medical prediction tasks.